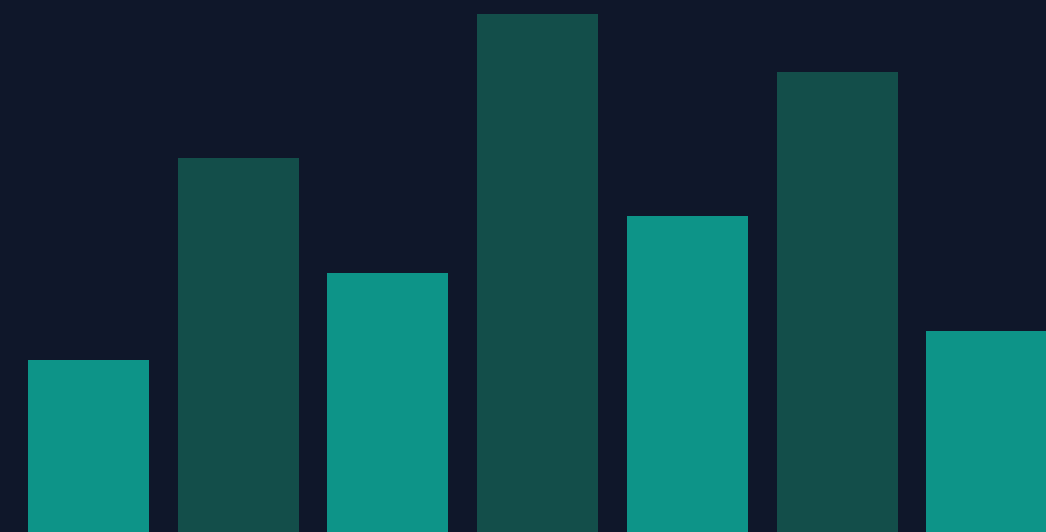


GROUP 8: City Skyline

การหา Next Greater Element ด้วย Monotonic Stack

การเปรียบเทียบอัลกอริทึม Next Greater Element

Brute Force vs Monotonic Stack



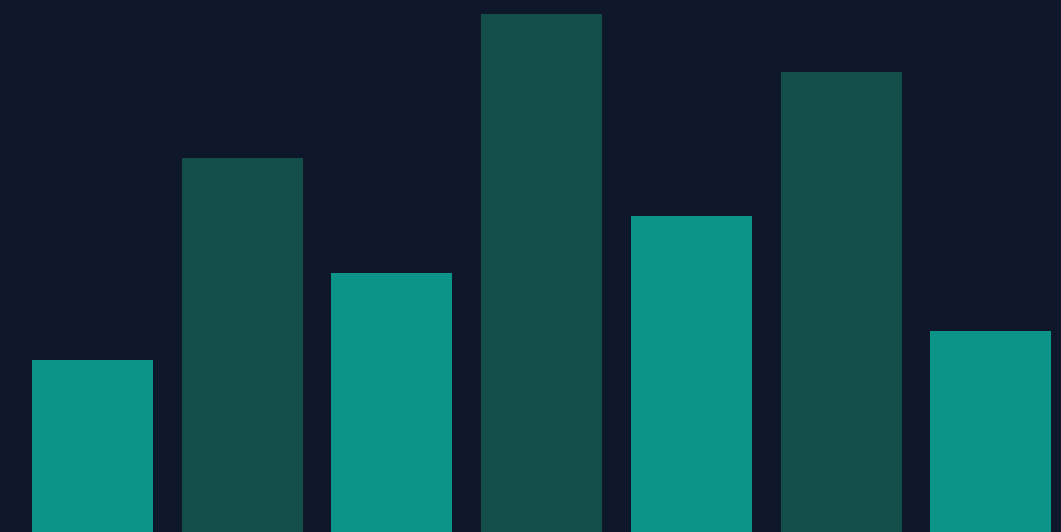
สมาชิกในกลุ่ม

68122250002 นายภูวเดช ตรังครนสิน

68122250018 นางสาวกิตติมาภรณ์ รถทอง

68122250025 นายธนารักษ์ จุลแดง

68122250024 นายอาลียะห์ อาลี



ภาพรวมโปรแกรม

โปรแกรมสาธิตและเปรียบเทียบอัลกอริทึม ประกอบด้วย 3 ไฟล์ที่ทำงานร่วมกัน

โปรแกรมโต้ตอบ

Main.java

รับความสูงตึกจากผู้ใช้ ตรวจสอบความถูกต้องของ input รันทั้ง 2 อัลกอริทึม พร้อมเทียบเวลาและผลลัพธ์

แกนอัลกอริทึม

NGEAlgorithm.java

เก็บ 3 เมธอดหลัก: bruteForceNGE, monotonicStackNGE และ largestRectangle

เบนช์มาร์กอัตโนมัติ

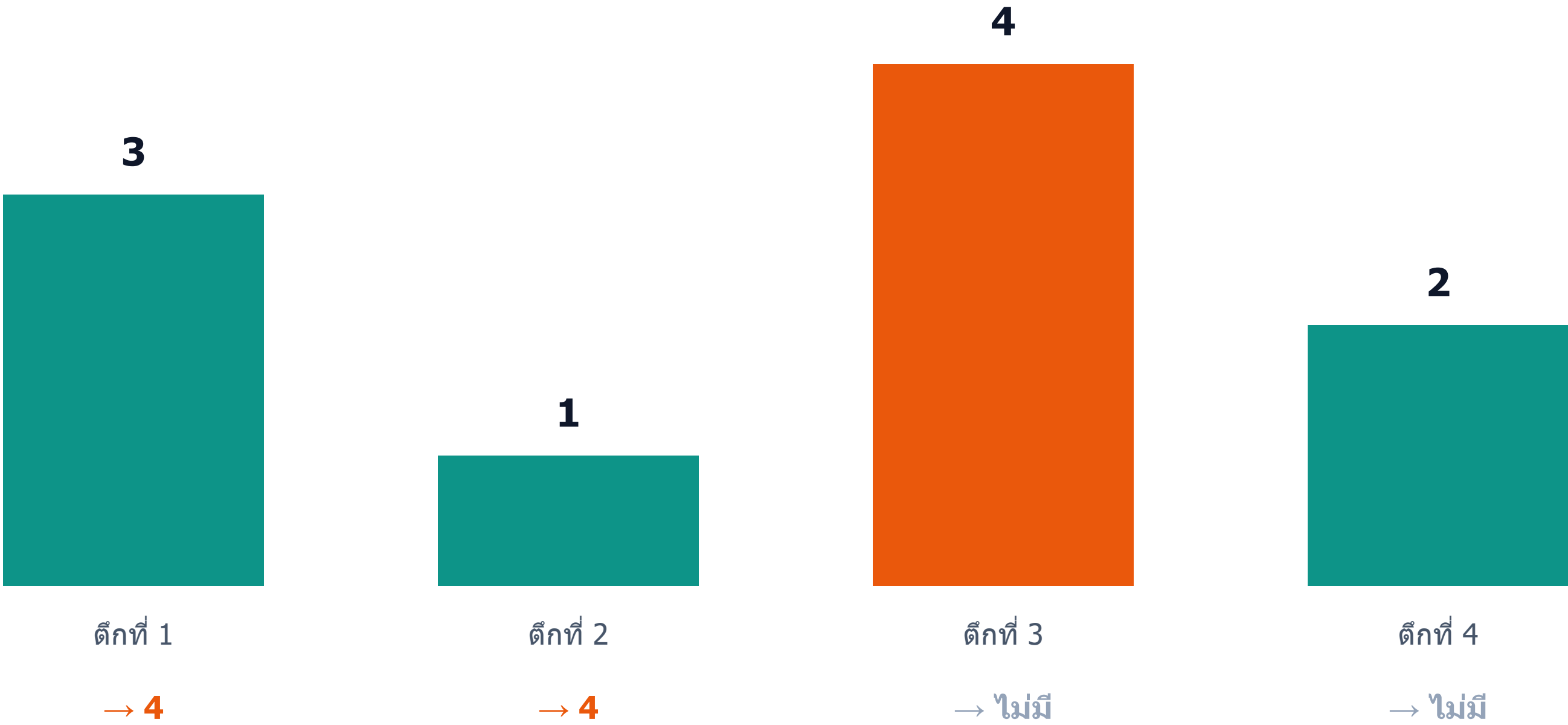
Experiment.java

ทดลองกับข้อมูลสุ่ม 5 ขนาด (10–10,000) รอบละ 5 ครั้ง หาค่าเฉลี่ยเวลาอย่างเป็นระบบ

โจทย์: Next Greater Element

สำหรับตึกแต่ละหลัง หา "ตึกถัดไปทางขวาที่สูงกว่า" — ถ้าไม่มีให้ตอบ -1

ตัวอย่าง: ความสูงตึก [3, 1, 4, 2] — ตึกที่ 1 และตึกที่ 2 ต่างมี **ตึกที่ 3 (สูง 4)** เป็นตึกถัดไปที่สูงกว่า



สองวิธีคิดที่ต่างกัน

วิธีที่ 1

Brute Force

"คนขี้เกียจแต่ตรงไปตรงมา"

เดินสำรวจตึกถัดไปที่ละหลัง ๆ จนกว่าจะเจอหลังที่สูงกว่า ทำซ้ำแบบนี้กับทุกตึก

ยิ่งตึกเยอะ ยิ่งต้องเดินสำรวจซ้ำมาก → ซ้ำแบบก้าวกระโดด

ความเร็ว: $O(n^2)$ — ช้าเมื่อข้อมูลใหญ่

วิธีที่ 2

Monotonic Stack

"คนฉลาดที่จำสิ่งที่เจอมาแล้ว"

เดินผ่านตึกแค่รอบเดียว ถือ "กองบันทึก" ไว้ในมือ — เจอตึกสูงกว่าเมื่อไหร่ก็รู้คำตอบทันที ไม่ต้องย้อนกลับไปสำรวจซ้ำ

แต่ละตึกถูกจัดการแค่ครั้งเดียว → เร็วแบบคงที่ต่อหน่วยข้อมูล

ความเร็ว: $O(n)$ — เร็วสม่ำเสมอ

ทั้งสองวิธีให้คำตอบถูกต้องเหมือนกันทุกประการ ต่างกันแค่ "ความเร็ว"

Main.java — รับและตรวจสอบ Input

ตัวอย่าง: การขอจำนวนอาคาร n จากผู้ใช้ ต้องเป็นจำนวนเต็มบวกเท่านั้น

1

อ่านค่า

scanner.nextInt() รับตัวเลขที่ผู้ใช้พิมพ์ เก็บใส่ n

2

ตรวจสอบ

ถ้า $n \leq 0 \rightarrow$ แจ้งเตือน แล้ว continue กลับไปถามใหม่

3

ผ่านเงื่อนไข

ถ้า $n > 0 \rightarrow$ break ออกจากลูป ไปขั้นตอนถัดไป

เปรียบเทียบ: เหมือนพนักงานถามลูกค้าว่า "จองกี่ที่?" ถ้าตอบไม่สมเหตุผล จะถามซ้ำจนกว่าจะได้คำตอบที่ใช้ได้



Main.java

```
n = scanner.nextInt();

if (n <= 0) {
    System.out.println(
        "Number of buildings
        must be greater than 0.");
    continue;
}

break;

// วนถามความสูงแต่ละตึกด้วย
// รูปแบบตรวจสอบเดียวกัน
if (height < 0) {
    System.out.println(
        "Height must not
        be negative.");
    continue;
}
```

NGEAlgorithm.java — สามเมธอดหลัก

$O(n^2)$

bruteForceNGE()

วนซ้อน 2 ชั้น หาไปทางขวาทีละตัวจนเจอค่ามากกว่า

```
for (int i = 0; i < n; i++) {
    for (int j = i+1; j < n; j++) {
        operationA++;
        if (heights[j] > heights[i]) {
            result[i] = heights[j];
            break;
        }
    }
}
```

$O(n)$

monotonicStackNGE()

ใช้ stack เก็บ index ที่ยังไม่เจอคำตอบ

```
while (!stack.isEmpty()) {
    if (heights[i] >
        heights[stack.peek()]) {
        int idx = stack.pop();
        result[idx] = heights[i];
    } else break;
}
stack.push(i);
```

Bonus

largestRectangle()

หาพื้นที่สี่เหลี่ยมใหญ่สุดในฮิสโตแกรม

```
while (!stack.isEmpty() &&
    curH < heights[stack.peek()]) {
    int h = heights[stack.pop()];
    int w = stack.isEmpty()
        ? i : i - stack.peek() - 1;
    maxArea = Math.max(maxArea, h*w);
}
```

Experiment.java — วิธีการทดลองวัดผล

ทดลองอย่างเป็นระบบเพื่อพิสูจน์ทฤษฎี Big-O ด้วยข้อมูลจริง

1 ข้อมูลสุ่ม (seed คงที่)

สุ่มความสูงตึก 1–1000 ด้วย Random(42) เพื่อให้ผลทดลองทำซ้ำได้เหมือนเดิมทุกครั้ง

2 Warm-up 3 รอบ

รันทั้งก่อน 3 ครั้งให้ JVM ปรับตัว (JIT) ป้องกันผลวัดคลาดเคลื่อนจากรอบแรก ๆ

3 ทดลองจริง 5 รอบ

จับเวลาแต่ละรอบด้วย nanoTime() แล้วหาค่าเฉลี่ย เพื่อความแม่นยำ

4 ขนาดข้อมูล 5 ระดับ

$n = 10, 100, 1,000, 5,000, 10,000$ — ไล่จากเล็กไปใหญ่เพื่อดูแนวโน้ม

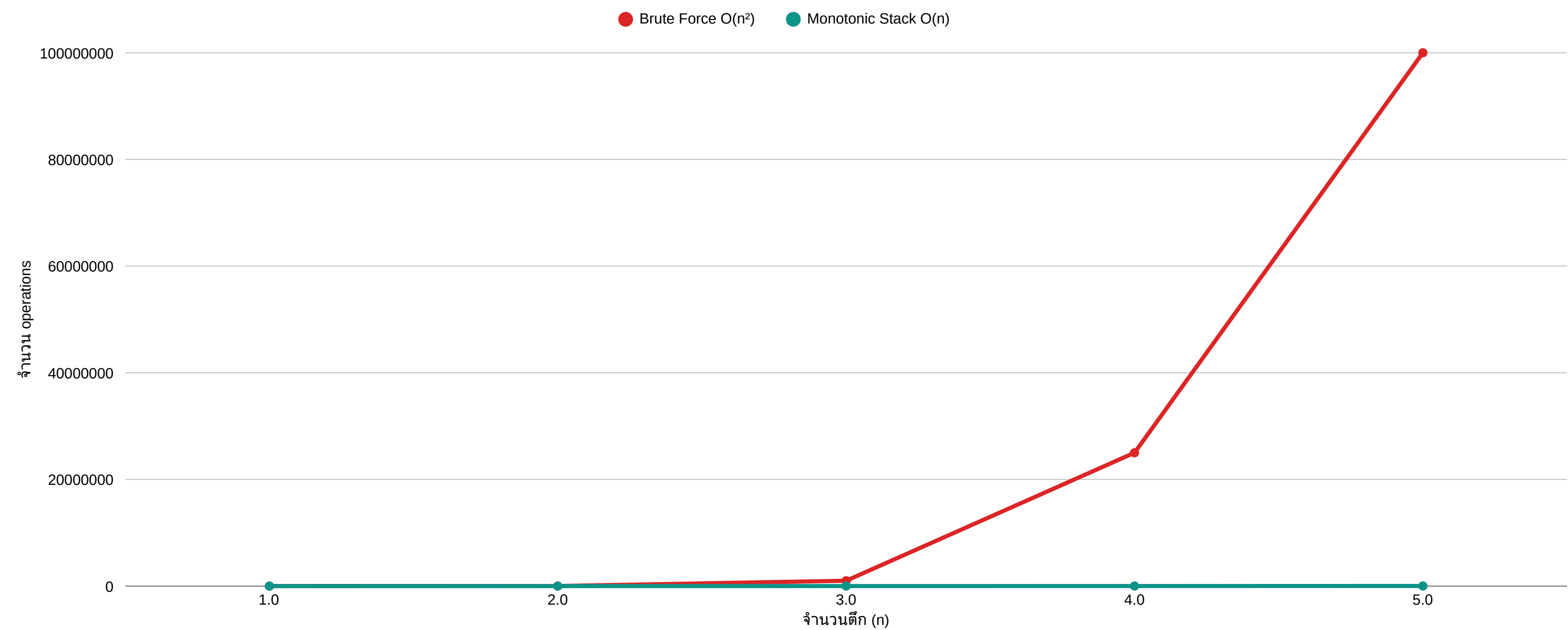


Experiment.java

```
for (int round = 1; round <= numberOfRounds; round++) {  
    long startA = System.nanoTime();  
    resultA = algorithm.bruteForceNGE(heights);  
    totalTimeA += (System.nanoTime() - startA); // สะสมเวลา  
}  
long averageTimeA = totalTimeA / numberOfRounds; // ค่าเฉลี่ย
```

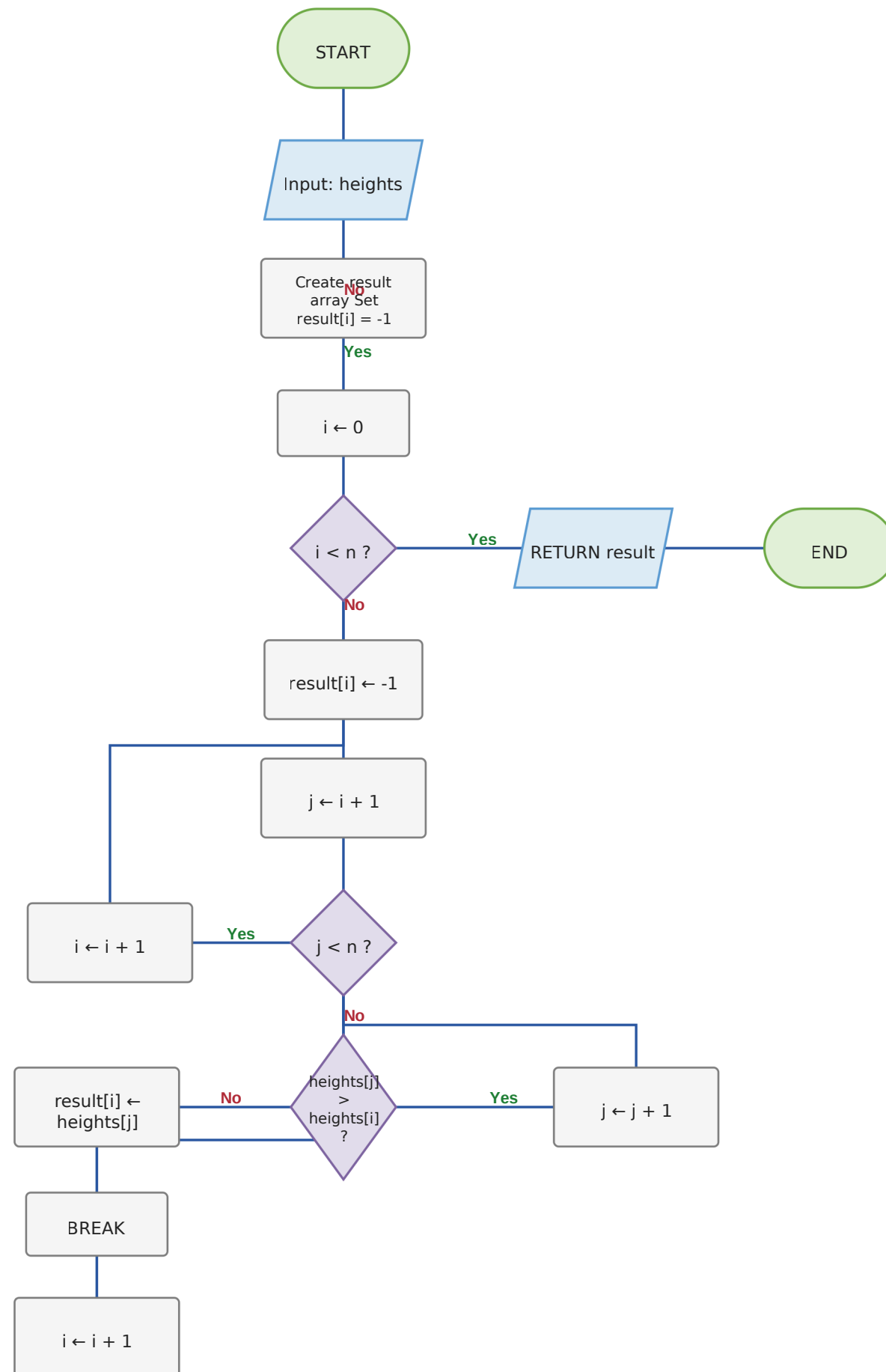

ผลที่คาดว่าจะเห็น: $O(n)$ vs $O(n^2)$

จำนวนการดำเนินการ (operations) เทียบกับจำนวนดีก n — ยิ่ง n มาก ความต่างยิ่งชัดเจน



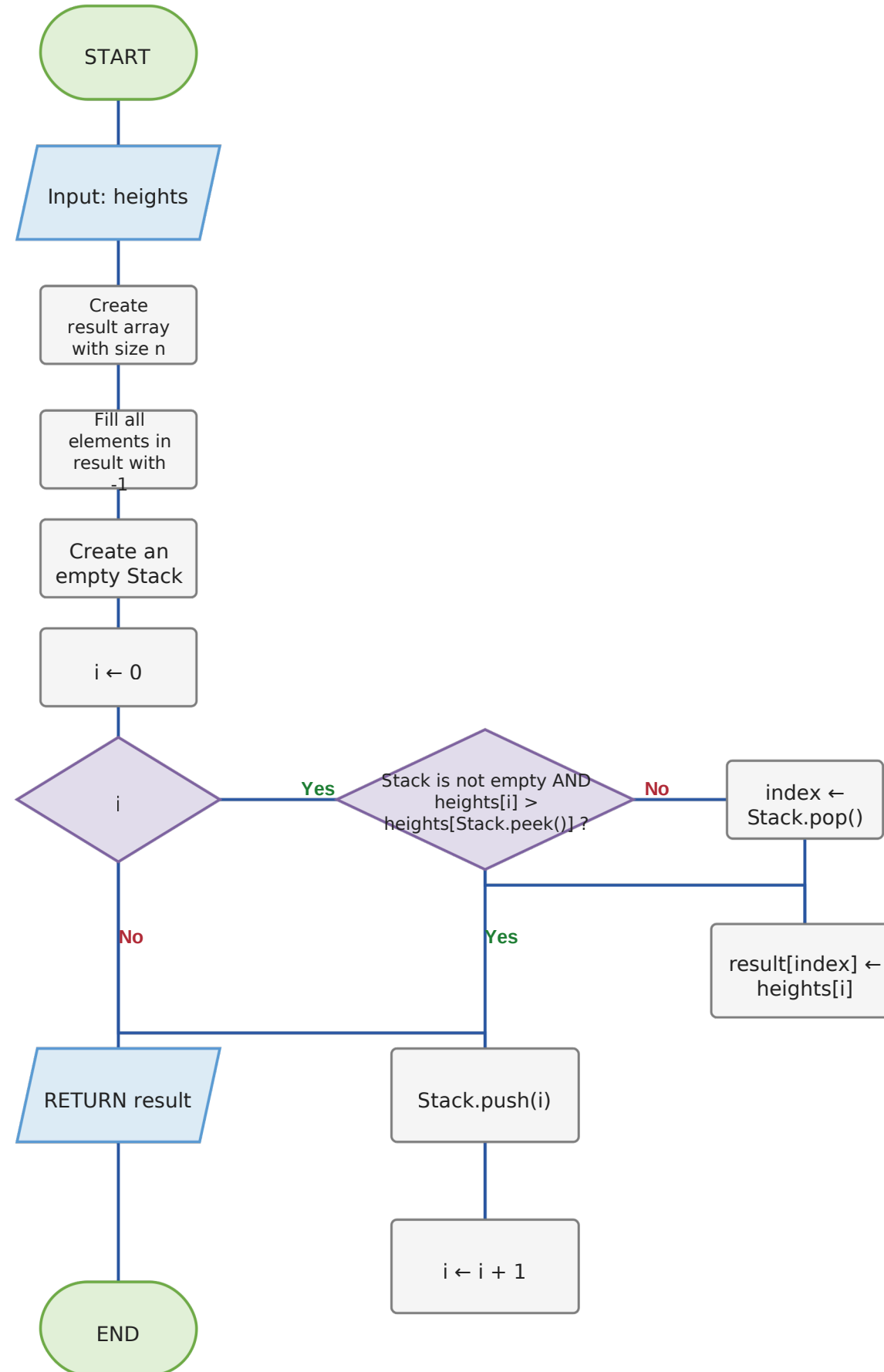
Flowchart A - Brute Force / Nested Loop

City Skyline - Next Greater Element



Flowchart B - Monotonic Stack

City Skyline - Next Greater Element



สรุป: โปรแกรมนี้คืออะไร

โปรแกรมสาธิตและเบนช์มาร์กเชิงวิทยาศาสตร์ (educational demo) ที่พิสูจน์ทฤษฎี Big-O ด้วยข้อมูลจริง ไม่ใช่โปรแกรมใช้งานทั่วไป

- โจทย์เดียวกันแก้ได้หลายวิธี — Brute Force เทียบกับ Monotonic Stack
- อัลกอริทึมที่ฉลาดกว่า (ใช้ stack) เร็วขึ้นมากเมื่อข้อมูลใหญ่ขึ้น
- พิสูจน์ด้วยการจับเวลาและนับ operations จริง ไม่ใช่แค่ทฤษฎีลอย ๆ
- ตรวจสอบผลลัพธ์ทั้งสองวิธีว่าตรงกัน (PASS/FAIL) เพื่อยืนยันความถูกต้อง